

Project SRS
Software Requirement Specification

[Real Time Chat Application]



Supervisor

[Dr. Ahmad Mustafa]

Submitted By

[Group Id: SE-23-03]

Students Name	Roll Number
Muhammad Abu Bakar	BSSE51F23S036
Tehzeeb Shahid	BSSE51F23S048
Mir Mohammad	BSSE51F22S046(RE)

Department of Software Engineering
University of Sargodha

Table of Contents

Table of Contents	ii
Revision History	iii
1.	Introduction 1
1.1	Purpose 1
1.2	Document Conventions 1
1.3	Intended Audience and Reading Suggestions 1
1.4	Product Scope 1
1.5	References 1
2.	Overall Description 1
2.1	Product Perspective 1
2.2	Product Functions 2
2.3	User Classes and Characteristics 2
2.4	Operating Environment 2
2.5	Design and Implementation Constraints 2
2.6	User Documentation 2
2.7	Assumptions and Dependencies 2
3.	External Interface Requirements 3
3.1	User Interfaces 3
3.2	Hardware Interfaces 3
3.3	Software Interfaces 3
3.4	Communications Interfaces 3
4.	System Features 3
4.1	System Feature 1 3
4.2	System Feature 2 (and so on) 4
5.	Other Nonfunctional Requirements 4
5.1	Performance Requirements 4
5.2	Safety Requirements 4
5.3	Security Requirements 4
5.4	Software Quality Attributes 5
5.5	Business Rules 5
6.	Other Requirements 5

1. Introduction

1.1 Purpose

This Software Requirement Specification (SRS) defines the functional and non-functional requirements of the **Real-Time Chat Application**, a web-based communication platform enabling users to exchange messages instantly over the internet. The document serves as a reference for developers, project supervisors, and stakeholders to understand system behavior, constraints, and expected functionality.

1.2 Document Conventions

This document follows the IEEE SRS documentation structure. Functional requirements are uniquely identified using requirement identifiers (e.g., REQ-01). Requirements are written in a clear and verifiable format using the keyword “**shall.**”

1.3 Intended Audience and Reading Suggestions

This document is intended for software developers, testers, project supervisors, and system designers. The introduction and overall description sections provide context, while later sections describe detailed functional and technical requirements.

1.4 Product Scope

The Real-Time Chat Application provides a web-based messaging platform allowing authenticated users to exchange messages instantly using WebSocket-based communication. The system demonstrates core communication functionality similar to modern messaging platforms such as WhatsApp while focusing on real-time web communication and structured backend architecture.

1.5 References

1. Django Documentation – <https://docs.djangoproject.com>
2. Django Channels Documentation – <https://channels.readthedocs.io>
3. Mozilla Developer Network WebSocket API Documentation

2. Overall Description

2.1 Product Perspective

The system is a standalone web application designed to support real-time text communication between authenticated users. It follows a client-server architecture where the frontend interface communicates with a Django-based backend server responsible for message processing, authentication, and database management.

2.2 Product Functions

The system will provide the following core functionalities:

- User account registration and authentication
- Real-time messaging between users
- Storage and retrieval of chat messages
- Display of active users
- Message timestamping and chat history

2.3 User Classes and Characteristics

Registered Users

- Primary system users
- Able to register, authenticate, and communicate through the chat interface
- Expected to possess basic computer and web navigation skills

System Administrator (Optional)

- Responsible for system monitoring and database management
- Possesses technical knowledge of system operation

2.4 Operating Environment

The application will operate in the following environment:

- Client Interface: Web browser (Chrome, Firefox, Edge)
- Backend Framework: Django (Python)
- Real-time Communication: WebSocket via Django Channels
- Database: SQLite or PostgreSQL
- Development Platform: Windows/Linux/macOS

2.5 Design and Implementation Constraints

- Backend must be implemented using Django.
- Real-time communication must use WebSocket technology via Django Channels.
- Frontend must use HTML, CSS, and JavaScript.

- Development must conform to the academic timeline and project guidelines.

2.6 User Documentation

The system will include the following documentation:

- User guide for application usage
- Technical documentation for system architecture
- Project report and system design diagrams

2.7 Assumptions and Dependencies

- Users have stable internet connectivity.
- Required development tools and frameworks are available.
- The application will initially support a limited number of concurrent users.

3. External Interface Requirements

3.1 User Interfaces

The system shall provide the following graphical user interfaces:

- User registration interface
- Login authentication interface
- Chat dashboard displaying conversation threads
- Message input and display interface
- Online user list panel

Interfaces shall follow modern usability principles and responsive web design.

3.2 Hardware Interfaces

The system requires standard computing hardware including a personal computer or mobile device with internet connectivity and a compatible web browser.

3.3 Software Interfaces

The application interfaces with:

- Django backend server

- Relational database management system
- Web browser rendering engine

3.4 Communications Interfaces

Communication between the client and server will utilize:

- HTTP protocol for standard web communication
- WebSocket protocol for real-time message exchange

4. System Features

4.1 User Authentication

4.1.1 Description and Priority

The system shall allow users to create accounts and securely authenticate to access chat services.

Priority: **High**

4.1.2 Stimulus/Response Sequences

User submits registration form →
System validates input →
Account is created and stored in database.

User submits login credentials →
System authenticates credentials →
User is granted access to the chat interface.

4.1.3 Functional Requirements

REQ-01: The system shall allow new users to register an account.

REQ-02: The system shall authenticate registered users using login credentials.

REQ-03: The system shall restrict system access to authenticated users only.

4.2 Real-Time Messaging

4.2.1 Description and Priority

The system shall enable instant communication between users using real-time messaging.

Priority: **High**

4.2.2 Stimulus/Response Sequences

User composes message →

Message transmitted via WebSocket →

Server processes message →

Recipient receives message instantly.

4.2.3 Functional Requirements

REQ-04: The system shall allow users to send messages to other users.

REQ-05: The system shall display received messages without page refresh.

REQ-06: The system shall store message data in the database for retrieval.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- Message delivery latency shall not exceed **2 seconds** under normal conditions.
- The system should support multiple simultaneous user connections.

5.2 Safety Requirements

The system must ensure message integrity and prevent data loss during transmission or storage.

5.3 Security Requirements

- All users must authenticate before accessing messaging features.
- User credentials shall be securely stored using encryption mechanisms.
- Unauthorized access to user data must be prevented.

5.4 Software Quality Attributes

Reliability: The system shall maintain stable operation with minimal downtime.

Usability: Interfaces shall be intuitive and user-friendly.

Maintainability: Source code shall follow modular design principles.

Scalability: System architecture should support future expansion.

5.5 Business Rules

- Only authenticated users may access messaging functionality.
- Each user account shall be uniquely identifiable within the system.

6. Other Requirements

The system shall maintain a database containing user accounts, message records, timestamps, and authentication information.

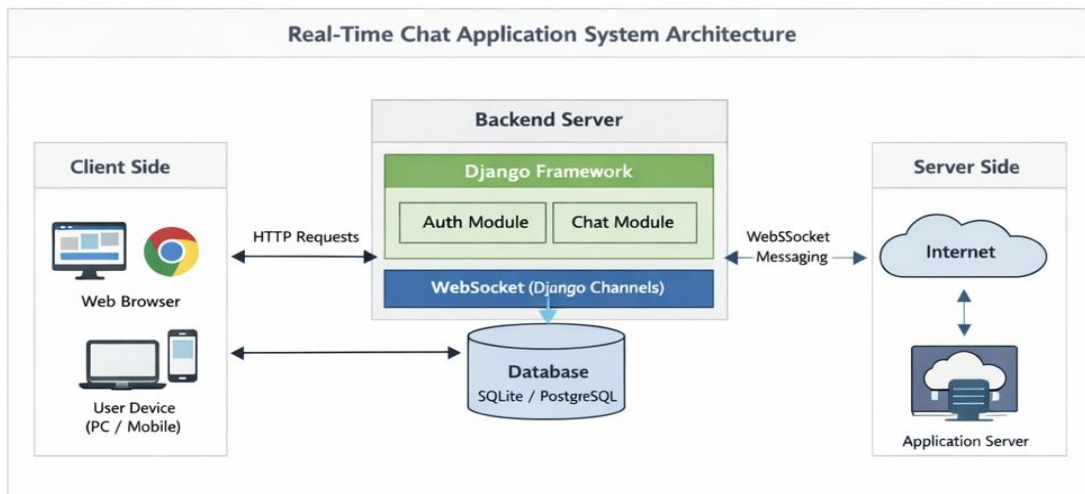
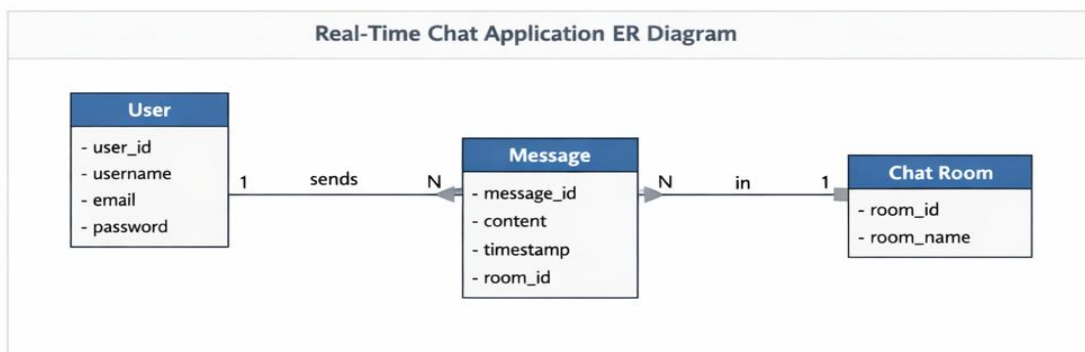
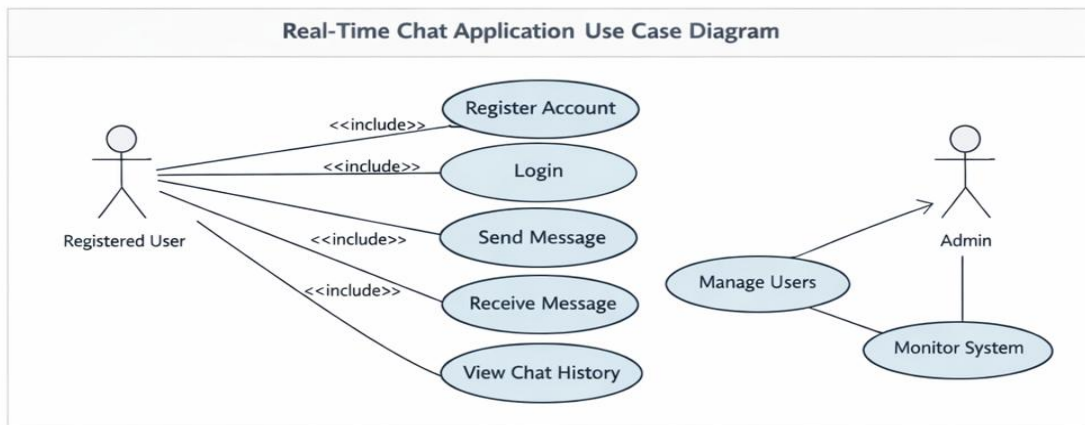
Appendix A: Glossary

Term	Definition
WebSocket	Protocol enabling real-time communication between client and server
Authentication	Process of verifying user identity
Real-Time Messaging	Instant message delivery without page refresh

Appendix B: Analysis Models

The system design will include the following analysis models:

- Use Case Diagram
- Entity Relationship Diagram (ERD)
- System Architecture Diagram



(Note: The above image is AI Generated)